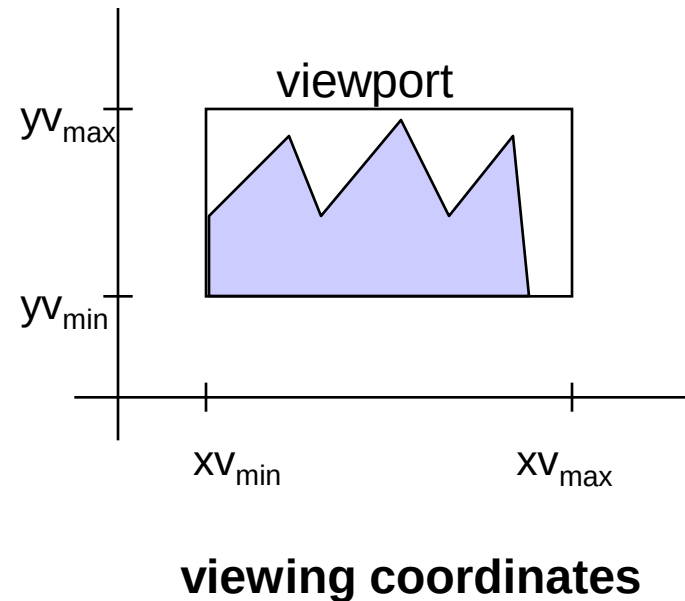
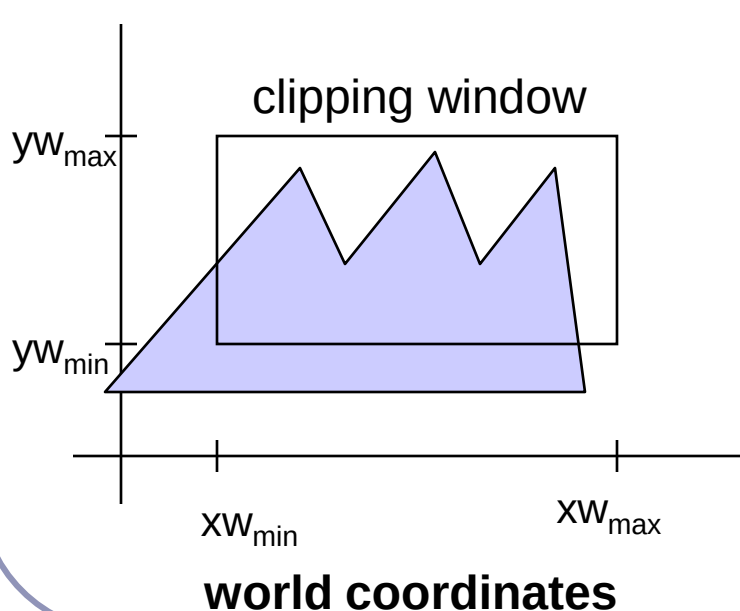


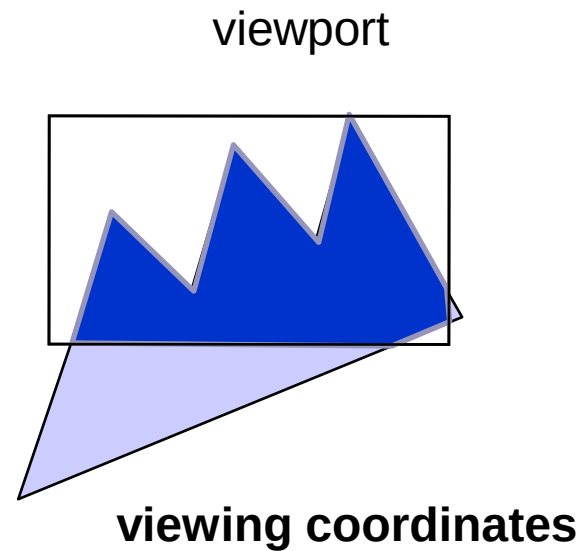
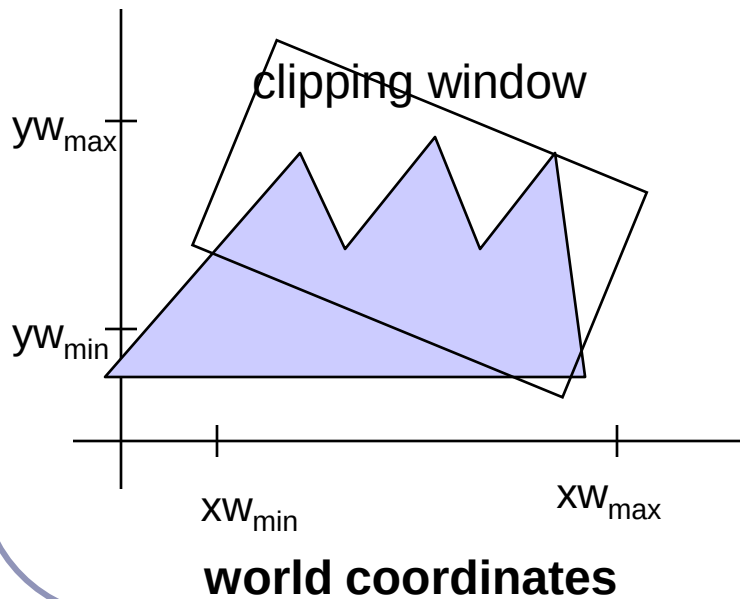
2D Viewing

2D Viewing

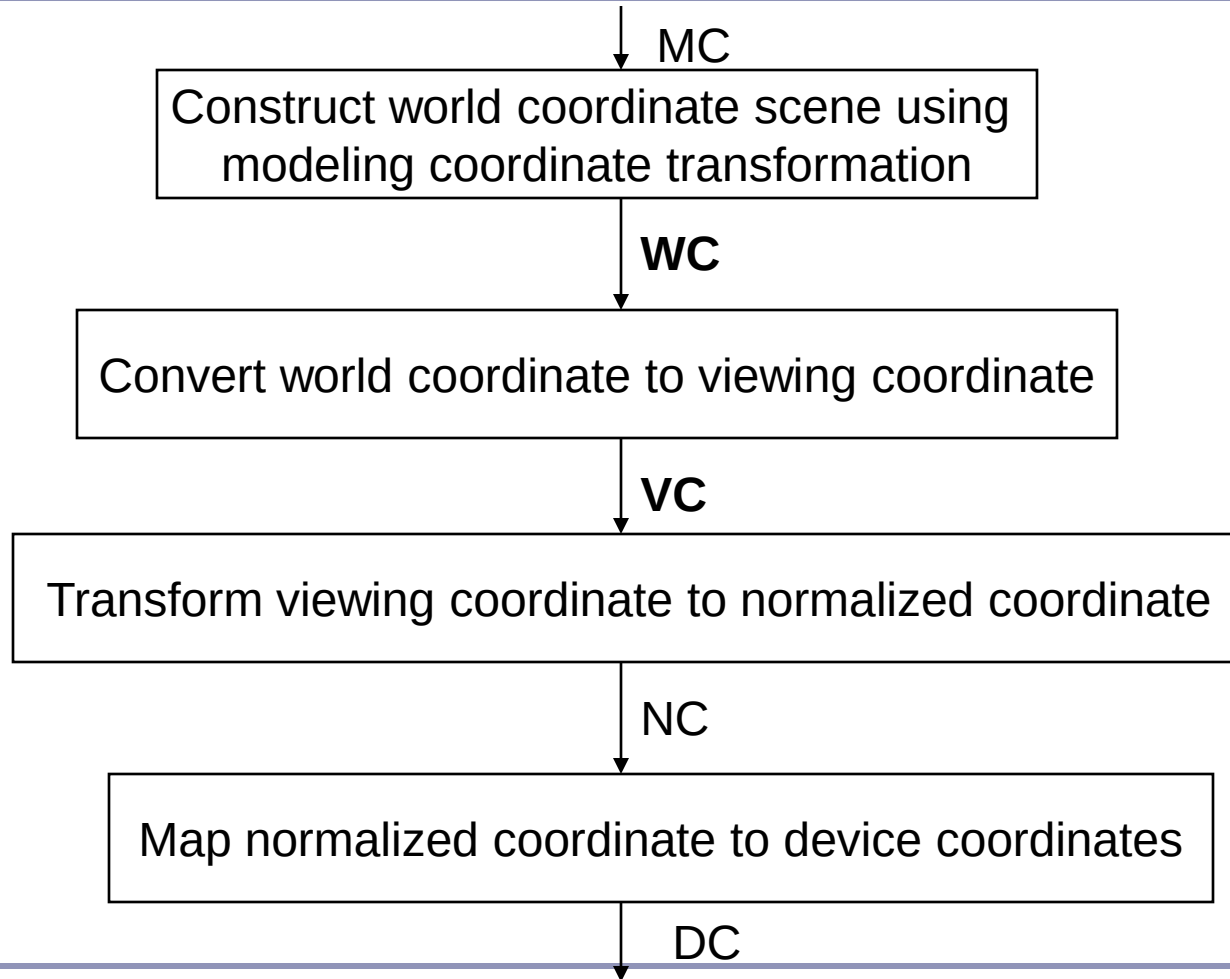
2D viewing transformation is a mapping of *world coordinate* scene to *device coordinates*



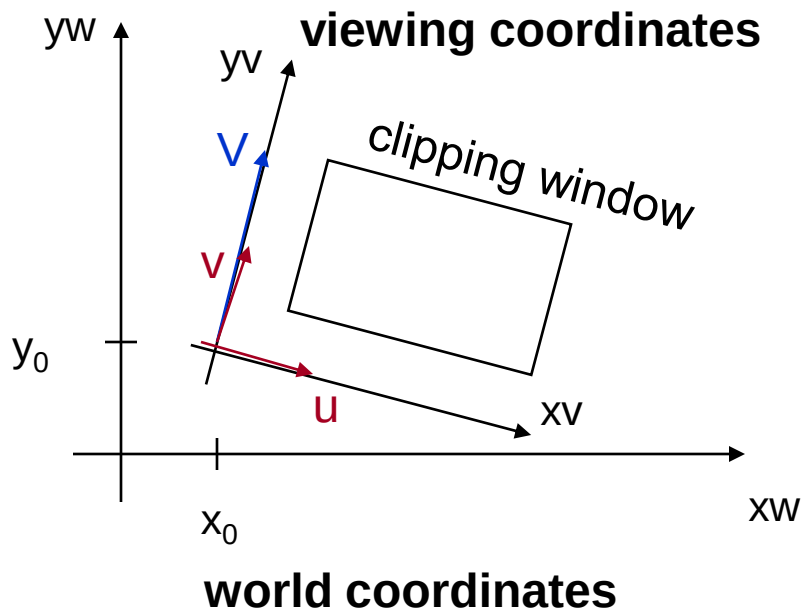
2D Viewing



2D Viewing Transformation Pipeline



2D Viewing



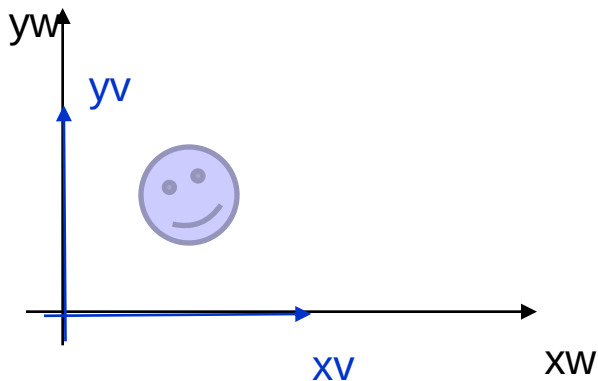
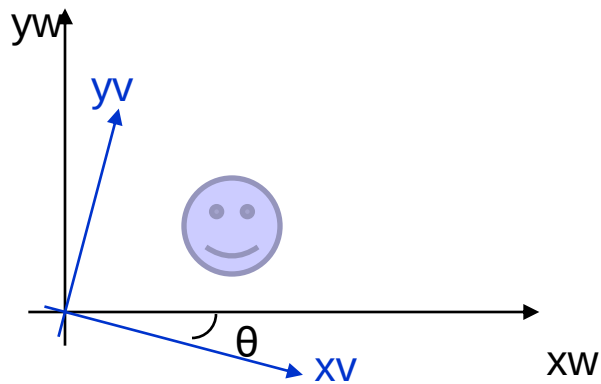
(x_0, y_0) is the viewing coordinate origin

V is view up vector

$v = (v_x, v_y)$ and

$u = (u_x, u_y)$ are unit vectors

2D Viewing

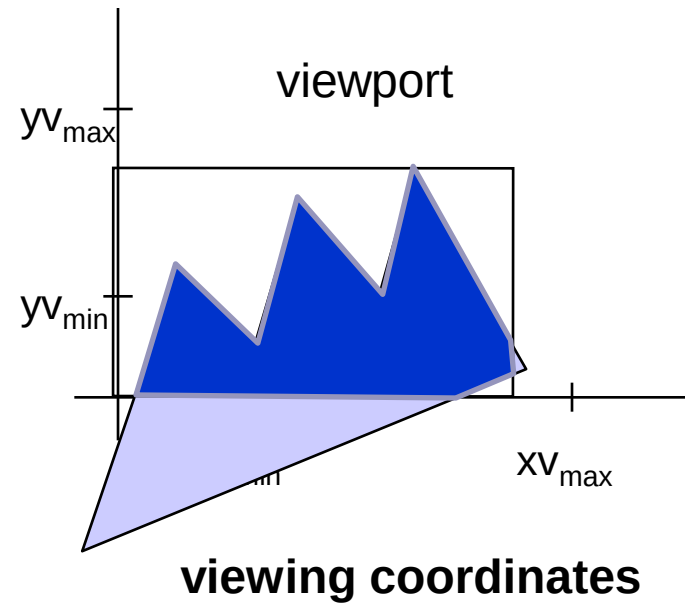
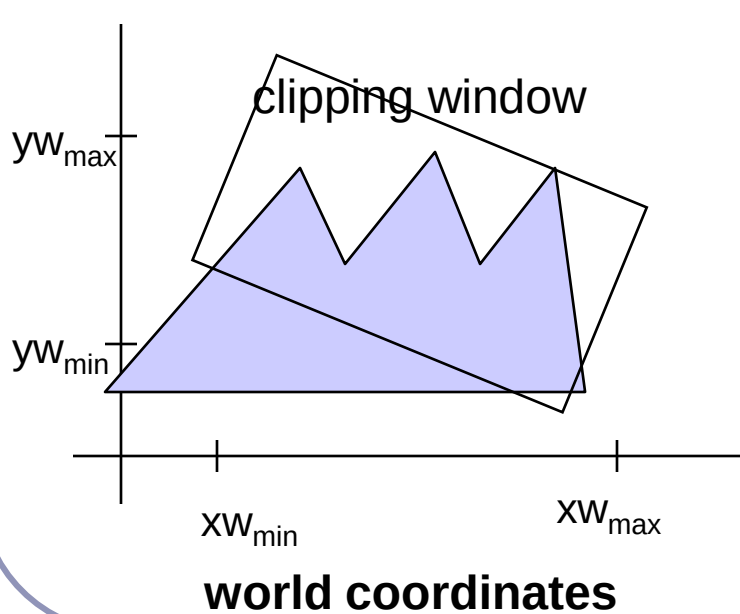


Transformation from the world coordinates to the viewing coordinates:

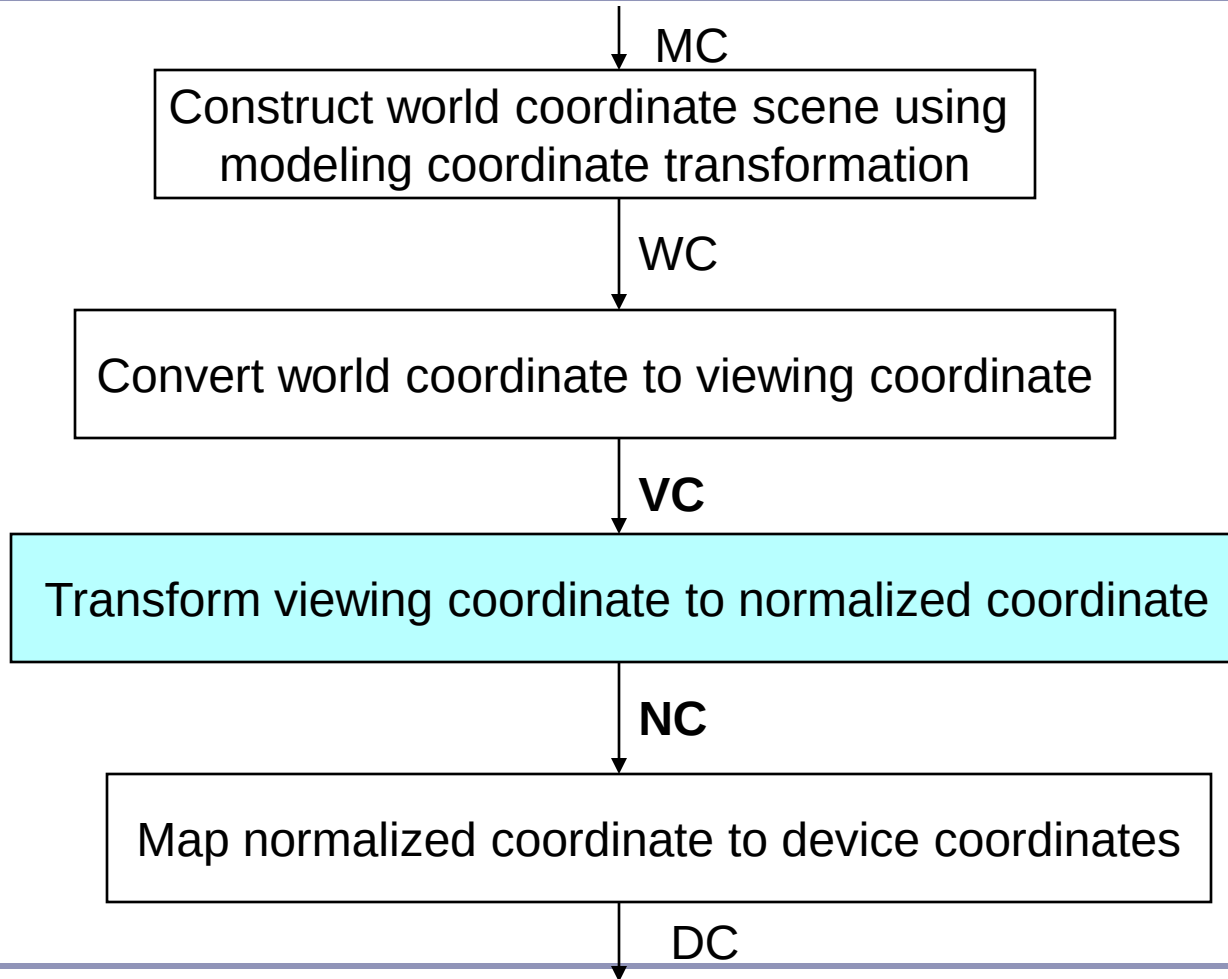
1. Translate viewing origin to world origin
2. Rotate viewing system to align it with the world frame

$$M_{wc,vc} = R(\theta) \cdot T(-x_0, -y_0)$$

2D Viewing

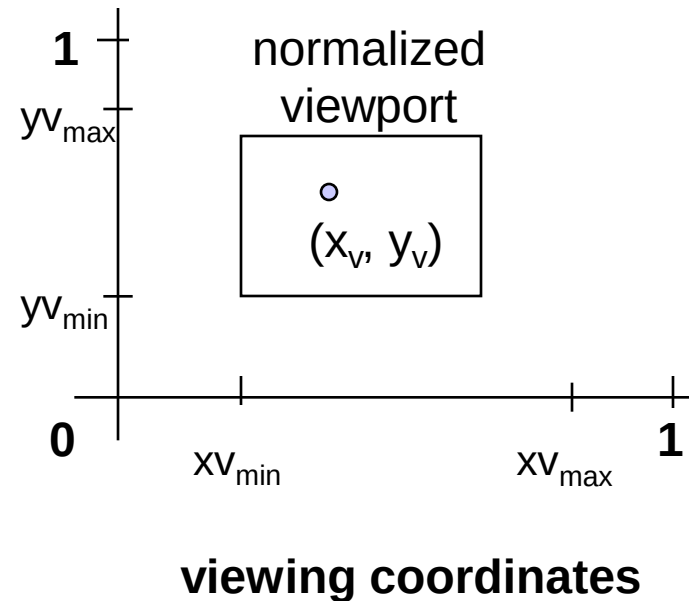
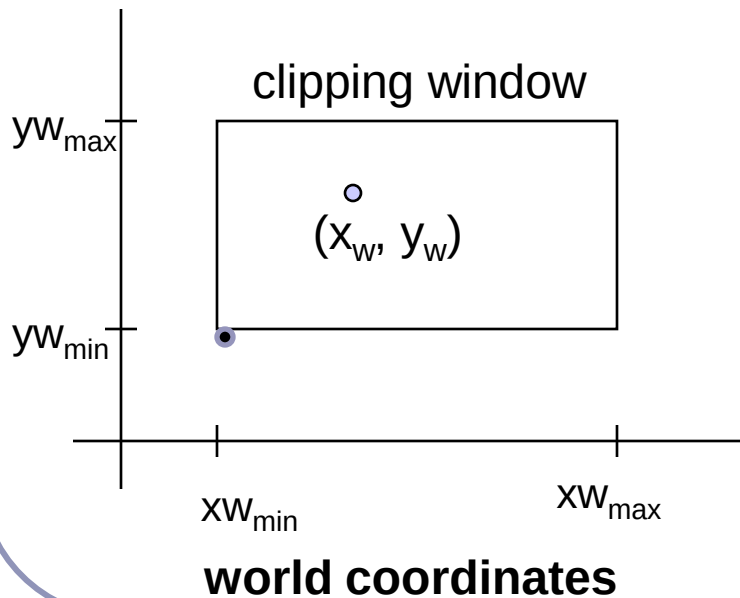


2D Viewing Transformation Pipeline

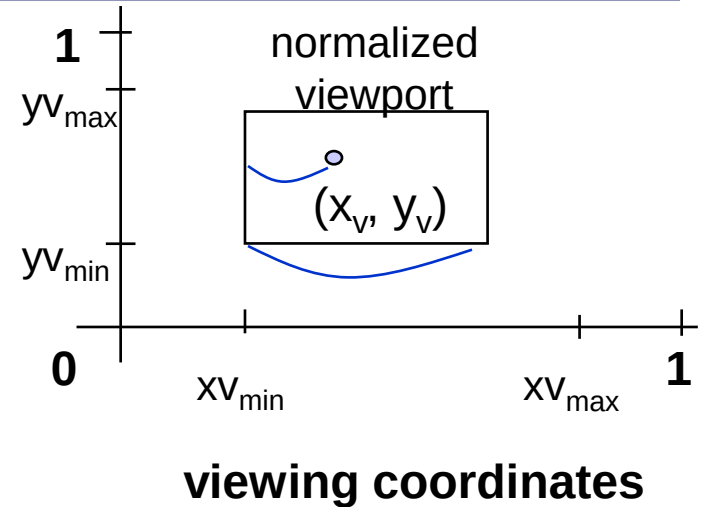
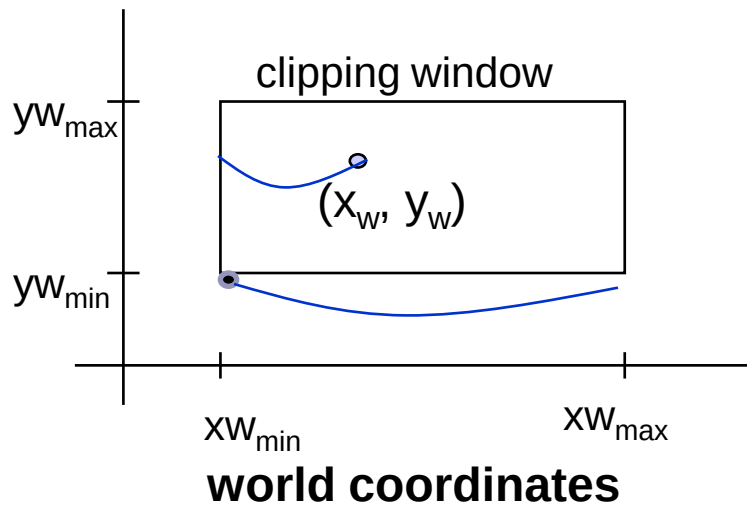


2D Viewing

Normalized Viewport



2D Viewing



$$\frac{x_w - xw_{min}}{xw_{max} - xw_{min}} = \frac{x_v - xv_{min}}{xv_{max} - xv_{min}}$$

$$x_v = s_x x_w + t_x$$

$$\frac{y_w - yw_{min}}{yw_{max} - yw_{min}} = \frac{y_v - yv_{min}}{yv_{max} - yv_{min}}$$

$$y_v = s_y y_w + t_y$$

2D Viewing

Normalized Viewport

$$M_{\text{window, normviewp}} = T \cdot S = \begin{bmatrix} s_x & 0 & t_x \\ 0 & s_y & t_y \\ 0 & 0 & 1 \end{bmatrix} \quad \text{where}$$

$$s_x = \frac{XV_{\max} - XV_{\min}}{XW_{\max} - XW_{\min}}$$

$$t_x = \frac{XW_{\max} XV_{\min} - XW_{\min} XV_{\max}}{XW_{\max} - XW_{\min}}$$

$$s_y = \frac{yV_{\max} - yV_{\min}}{y_{\max} - yw_{\min}}$$

$$t_y = \frac{yw_{\max} yV_{\min} - yw_{\min} yV_{\max}}{yw_{\max} - yw_{\min}}$$

2D Viewing

Normalized Viewport

1. Scale clipping window to the size of viewport using a fixed-point $(xw_{\min}, yw_{\min})$

$$S = T(-xw_{\min}, -yw_{\min}) \cdot S(s_x, s_y) \cdot T(-xw_{\min}, -yw_{\min})$$

2. Translate $(xw_{\min}, yw_{\min})$ to $(xv_{\min}, yv_{\min})$

$$T = T(xv_{\min} - xw_{\min}, yv_{\min} - yw_{\min})$$

$$M_{\text{window, normviewp}} = T \cdot S$$

OpenGL

glMatrixMode(GL_PROJECTION)

- selects the projection mode for constructing the matrix to transform from world coordinates to screen coordinates
- **gluOrtho2D (xwmin, xwmax, ywmin, ywmax)**
 - defines a 2D clipping window. 3D scene is projected along parallel lines that are perpendicular to xy display screen (orthogonal projection)
- **glViewport (xvmin, yvmin, vpWidth, vpHeight)**
 - specifies viewport parameters
- **glGetIntegerv (GL_VIEWPORT, vpArray)**
 - returns the parameters of the current viewport to vpArray
 - vpArray: 4-element array

OpenGL

- **glutInit (&argc, argv)**
 - initializes GLUT
- **glutInitWindowPosition (xTopLeft, yTopLeft)**
 - initializes display window position
- **glutInitWindowSize (dwWidth, dwHeight)**
 - initializes display window size
- **glutCreateWindow (“title”)**
 - creates the display window

OpenGL

- **glutInitDisplayMode (mode)**
 - used to choose color mode and buffering mode
 - GLUT_RGB: color mode
 - GLUT_SINGLE: buffering mode

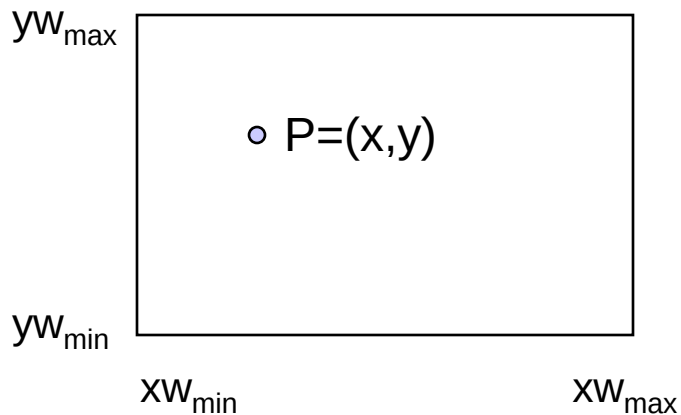
OpenGL

- **glutDisplayFunction (dispFunc)**
 - dispFunc: is the name of the routine that describes what is to be displayed
- **glutPostRedisplay ()**
 - used to renew the contents of the current display window
- **glutMainLoop ()**
 - GLUT processing loop

2D Clipping Algorithms

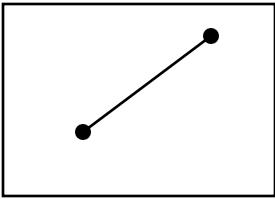
- Point Clipping
- Line Clipping
- Fill-area Clipping
- Curve Clipping
- Text Clipping

Point Clipping

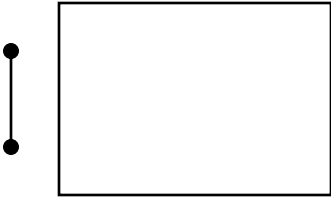


P will be displayed if
 $xw_{min} \leq x \leq xw_{max}$ and
 $yw_{min} \leq y \leq yw_{max}$

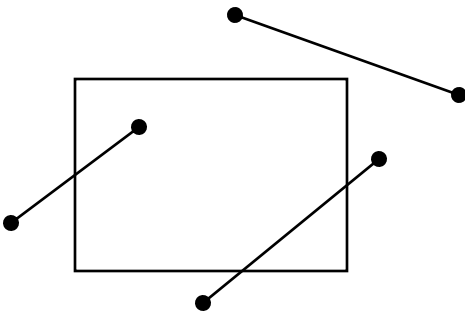
Line Clipping



If both endpoints are inside of all 4 clipping boundaries => **inside**

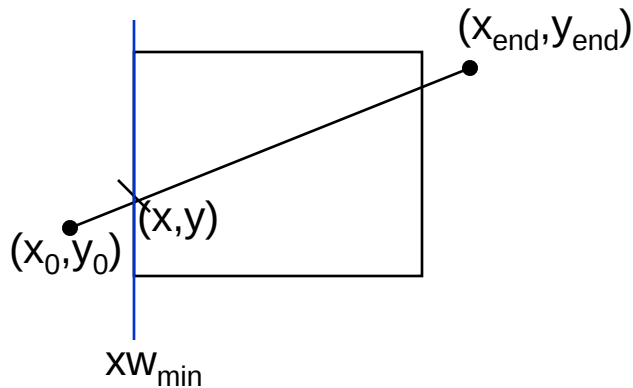


If both endpoints are outside any one of the 4 boundaries
=> **outside**



Otherwise, line intersects at least one boundary and it may or may not cross the window

Line Clipping



Finding the intersection point of the line and the window border

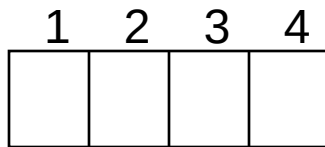
$$x = x_0 + u(x_{\text{end}} - x_0)$$

$$y = y_0 + u(y_{\text{end}} - y_0)$$

$$u = \frac{x - x_0}{x_{\text{end}} - x_0} = \frac{y - y_0}{y_{\text{end}} - y_0}$$

if $0 \leq u \leq 1$ part of the line is inside

Cohen-Sutherland Line Clipping



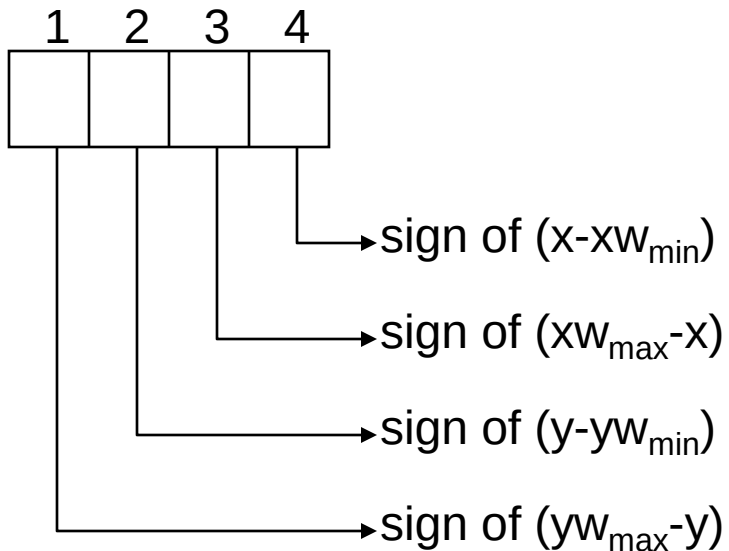
top bottom right left

1 - endpoint is outside of that window border

0 - inside or on the border

1 0 0 1	1 0 0 0	1 0 1 0
0 0 0 1	0 0 0 0	0 0 1 0
0 1 0 1	0 1 0 0	0 1 1 0

Cohen-Sutherland Line Clipping



A region-code 0000 for both endpoints
 $\Rightarrow$ completely inside

Region-code 1 in the same bit position
for each endpoint
 $\Rightarrow$ completely outside

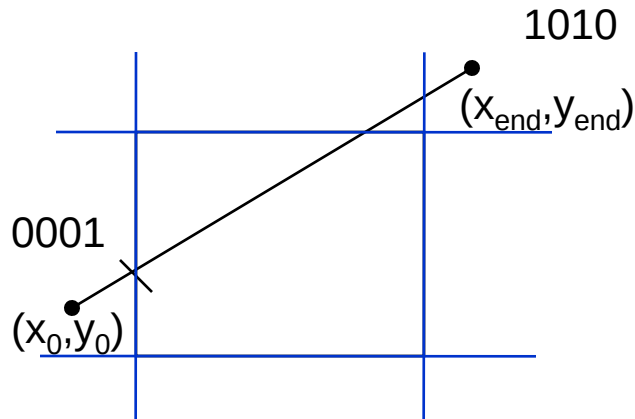
If OR of region codes of endpoints is
0000 $\Rightarrow$ inside

If AND of region codes of endpoints is
not 0000 $\Rightarrow$ outside

Other cases: check for intersection

Cohen-Sutherland Line Clipping

Ex:



$$0001 \vee 1010 = 1011$$

not 0000 $\rightarrow$ not inside

$$0001 \wedge 1010 = 0000$$

0000 $\rightarrow$ not outside

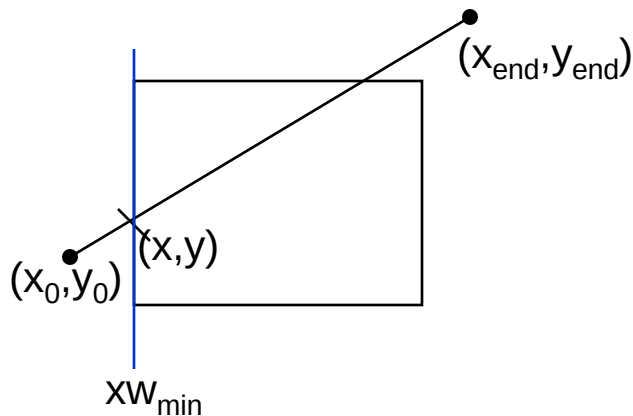
$\rightarrow$ Find intersection points

$$(x_0, y_0) = (1, 2)$$

$$(x_{\text{end}}, y_{\text{end}}) = (11, 12)$$

Cohen-Sutherland Line Clipping

Left border



$$x_{W_{min}} = 2$$

$$u = \frac{x - x_0}{x_{end} - x_0} = \frac{2 - 1}{11 - 1} = 0.10 \quad 0 \leq u \leq 1$$

$$y = y_0 + u(y_{end} - y_0) = 2 + 0.10(12 - 2) = 3$$

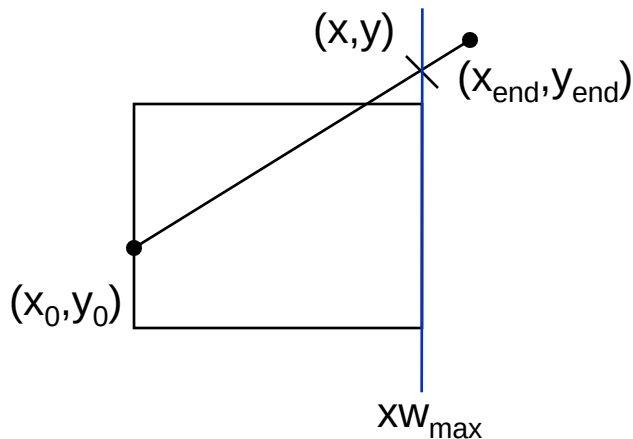
Intersection point: (2,3)

$$(x_0, y_0) = (1, 2)$$

$$(x_{end}, y_{end}) = (11, 12)$$

Cohen-Sutherland Line Clipping

Right border



$$xW_{max} = 10$$

$$u = \frac{x - x_0}{x_{end} - x_0} = \frac{10 - 2}{11 - 2} = 0.89 \quad 0 \leq u \leq 1$$

$$y = y_0 + u(y_{end} - y_0) = 2 + 0.89(12 - 2) \approx 11$$

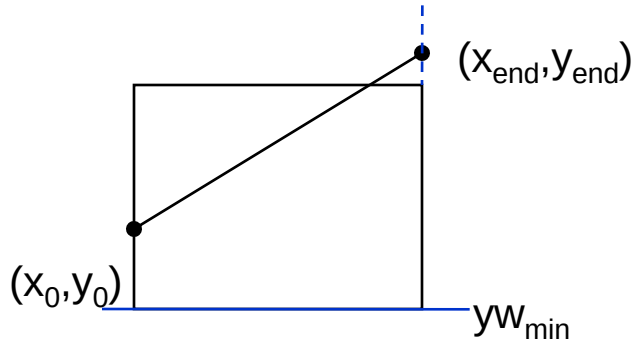
Intersection point: (10,11)

$$(x_0, y_0) = (2, 3)$$

$$(x_{end}, y_{end}) = (11, 12)$$

Cohen-Sutherland Line Clipping

Bottom border



$$yW_{min} = 1$$

$$u = \frac{y - y_0}{y_{end} - y_0} = \frac{1 - 3}{11 - 3} = -0.25$$

$$u < 0$$

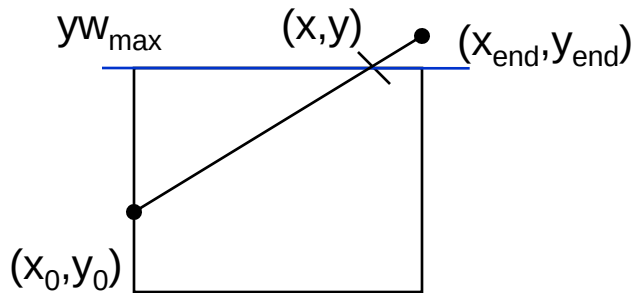
No intersection

$$(x_0, y_0) = (2, 3)$$

$$(x_{end}, y_{end}) = (10, 11)$$

Cohen-Sutherland Line Clipping

Top border



$$yW_{max} = 10$$

$$u = \frac{y - y_0}{y_{end} - y_0} = \frac{10 - 3}{11 - 3} = 0.875 \quad 0 \leq u \leq 1$$

$$x = x_0 + u(x_{end} - x_0) = 2 + 0.875(10 - 2) = 9$$

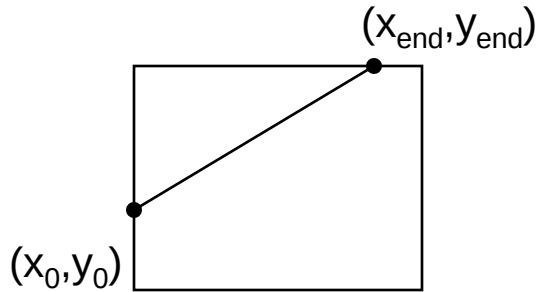
Intersection point: (9,10)

$$(x_0, y_0) = (2, 3)$$

$$(x_{end}, y_{end}) = (10, 11)$$

Cohen-Sutherland Line Clipping

Finally



$$(x_0, y_0) = (2, 3)$$

$$(x_{\text{end}}, y_{\text{end}}) = (9, 10)$$

```

inline GLint round (const GLfloat a)
    { return GLint(a+0.5); }

const GLint winLeftBitCode = 0x1;
const GLint winRightBitCode = 0x2;
const GLint winBottomBitCode = 0x4;
const GLint winTopBitCode = 0x8;

inline GLint inside (GLint code) { return GLint(!code); }

inline GLint reject (GLint code1, GLint code2)
    { return GLint(code1 & code2); }

inline GLint accept (GLint code1, GLint code2)
    { return GLint(!(code1 | code2)); }

GLubyte encode (wcPt2D pt, wcPt2D winMin, wcPt2D winMax)
{
    GLubyte code = 0x00;

    if (pt.x < winMin.x)
        code = code | winLeftBitCode;
    if (pt.x > winMax.x)
        code = code | winRightBitCode;
    if (pt.y < winMin.y)
        code = code | winBottomBitCode;
    if (pt.y > winMax.y)
        code = code | winTopBitCode;
    return (code);
}

void swapPts (wcPt2D * p1, wcPt2D * p2)
{
    wcPt2D tmp;

    tmp=*p1; *p1=*p2; *p2=tmp;
}

void swapCodes (GLubyte * c1, GLubyte * c2)
{
    GLubyte tmp;

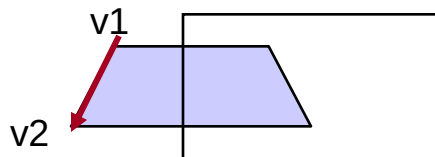
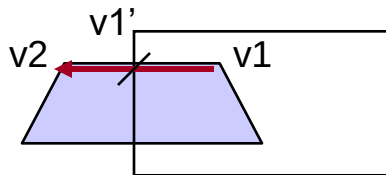
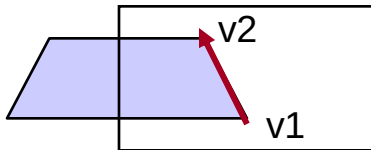
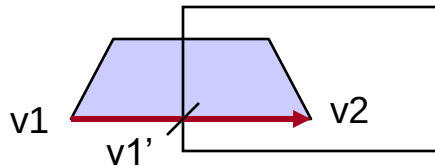
    tmp=*c1; *c1=*c2; *c2=tmp;
}

void lineClipCohSuth (wcPt2D winMin, wcPt2D winMax, wcPt2D p1, wcPt2D p2)
{
    GLubyte code1, code2;
    GLint done = false, plotLine = false;
    GLfloat m;

    while (!done) {
        code1 = encode (p1, winMin, winMax);
        code2 = encode (p2, winMin, winMax);
        if (accept (code1, code2)) {
            done = true;
            plotLine = true;
        }
        else
            if (reject (code1, code2))
                done = true;
            else {
                /* Label the endpoint outside the display window as p1. */
                if (inside (code1)) {
                    swapPts (&p1, &p2);
                    swapCodes (&code1, &code2);
                }
                /* Use slope m to find line-clipEdge intersection. */
                if (p2.x != p1.x)
                    m = (p2.y - p1.y) / (p2.x - p1.x);
                if (code1 & winLeftBitCode) {
                    p1.y += (winMin.x - p1.x) * m;
                    p1.x = winMin.x;
                }
                else
                    if (code1 & winRightBitCode) {
                        p1.y += (winMax.x - p1.x) * m;
                        p1.x = winMax.x;
                    }
                    else
                        if (code1 & winBottomBitCode) {
                            /* Need to update p1.x for nonvertical lines only. */
                            if (p2.x != p1.x)
                                p1.x += (winMin.y - p1.y) / m;
                            p1.y = winMin.y;
                        }
                        else
                            if (code1 & winTopBitCode) {
                                if (p2.x != p1.x)
                                    p1.x += (winMax.y - p1.y) / m;
                                p1.y = winMax.y;
                            }
                        }
            }
    }
    if (plotLine)
        lineBres (round (p1.x), round (p1.y), round (p2.x), round (p2.y));
}

```

Sutherland-Hodgman Polygon Clipping



1. First vertex is outside the window border and second vertex is inside

=> send the intersection point and the second vertex to the next clipper

2. Both vertices are inside

=> send only the second vertex

3. First vertex is inside and the second vertex is outside

=> send only the intersection point

4. Both vertices are outside

=> no vertices are sent

Sutherland-Hodgman Polygon Clipping

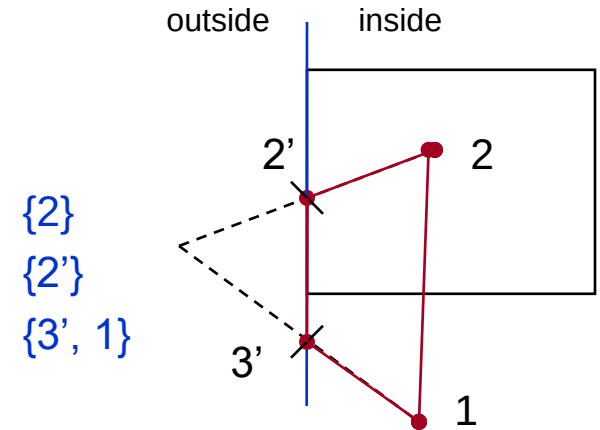
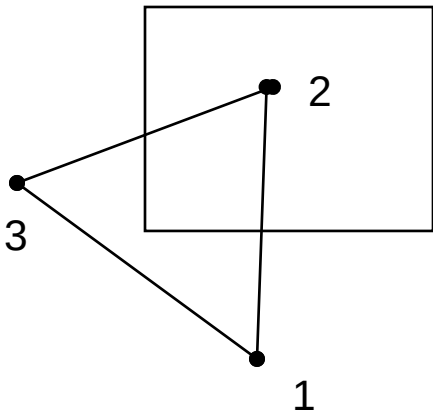
Ex:

Left border

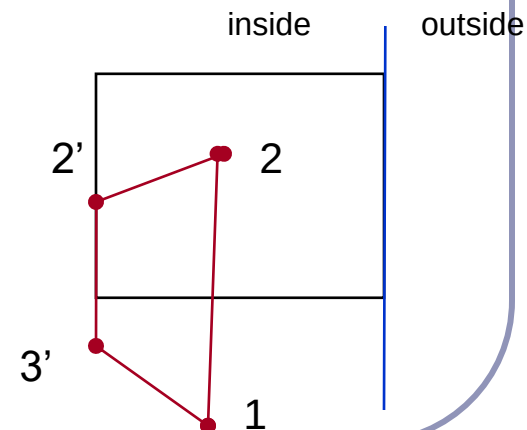
$1 \rightarrow 2$	Rule 2: inside $\rightarrow$ inside
$2 \rightarrow 3$	Rule 3: inside $\rightarrow$ outside
$3 \rightarrow 1$	Rule 1: outside $\rightarrow$ inside

Right border

$2 \rightarrow 2'$	Rule 2: inside $\rightarrow$ inside
$2' \rightarrow 3'$	Rule 2: inside $\rightarrow$ inside
$3' \rightarrow 1$	Rule 2: inside $\rightarrow$ inside
$1 \rightarrow 2$	Rule 2: inside $\rightarrow$ inside

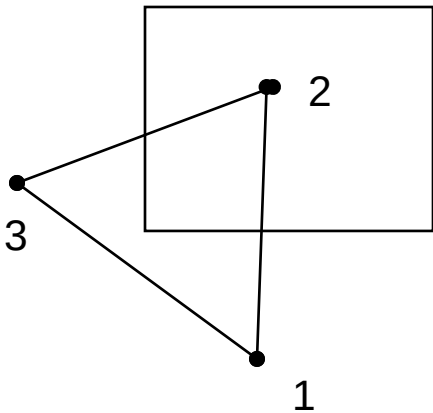


{2}
{2'}
{3', 1}



{2'}
{3'}
{1}
{2}

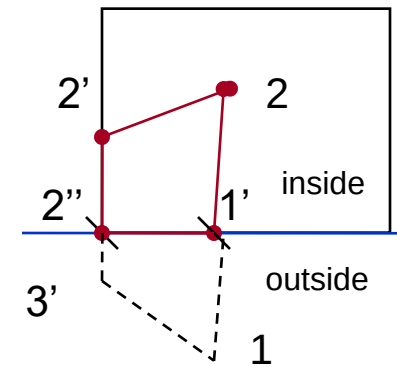
Sutherland-Hodgman Polygon Clipping



{2'} {3'} {1} {1}

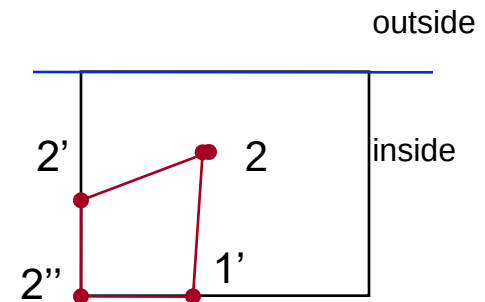
Bottom border

$2' \rightarrow 3'$	Rule 3: inside $\rightarrow$ outside	{2''}
$3' \rightarrow 1$	Rule 4: outside $\rightarrow$ outside	{}
$1 \rightarrow 2$	Rule 1: outside $\rightarrow$ inside	{1', 2}
$2 \rightarrow 2'$	Rule 2: inside $\rightarrow$ inside	{2'}

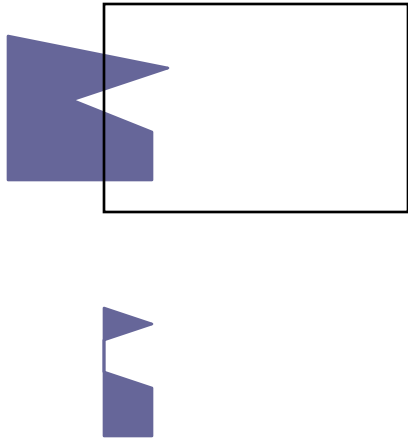


Top border

$2'' \rightarrow 1'$	Rule 2: inside $\rightarrow$ inside	{1'}
$1' \rightarrow 2$	Rule 2: inside $\rightarrow$ inside	{2}
$2 \rightarrow 2'$	Rule 2: inside $\rightarrow$ inside	{2'}
$2' \rightarrow 2''$	Rule 2: inside $\rightarrow$ inside	{2''}



Sutherland-Hodgman Polygon Clipping



Concave Polygons

- Split concave polygon into convex polygons and then use Sutherland-Hodgman algorithm

or

- Modify the algorithm to check the vertex list for multiple intersection points along any boundary. Then split into separate sections.

Algorithm on page 333